DHARAA Smart Irrigation System

Detailed Project Design, Features, Hardware, Datasets, AI/ML and Implementation

ESP32 • MicroPython • Flask • SQLite • HTML/CSS/JavaScript • Chart.js

How to use this guide

Follow the roadmap in order. Confirm electrical ratings before wiring the pump. Unavailable measurements are labelled rather than invented.

Contents

1. 1. Project overview
2. 2. Goals and user workflow
3. 3. Architecture
4. 4. Hardware connections and limitations
5. 5. Sensor calibration and MicroPython firmware
6. 6. Backend, database and APIs
7. 7. Dashboard features and implementation
8. 8. Irrigation rules and safety
9. 9. Datasets and external APIs
10. 10. Machine-learning workflow
11. 11. Project directory structure
12. 12. Step-by-step roadmap
13. 13. Testing checklist
14. 14. Deployment and maintenance
15. 15. Feature availability matrix

1. Project overview

DHARAA is an IoT smart-irrigation system for monitoring field conditions, controlling a water pump, planning irrigation, checking weather, providing crop advice and analysing historical sensor readings.

Adapted to the described kit: ESP32 with MicroPython, capacitive soil-moisture sensor, DHT22, rain sensor, water-flow sensor, relay module, 5V pump and adapter, LED, breadboard and jumper wires.

Do not invent unavailable values. The current kit cannot measure tank level, electricity consumption, NPK, pH or multiple soil depths. Mark these as “not monitored” until suitable hardware or trusted data is added.

2. Goals and user workflow

- Read soil moisture, temperature, humidity, rain state and flow pulses.
- Build a responsive web dashboard with device health, pump state and command history.
- Create farm profiles and irrigation schedules.
- Support manual and automatic pump operation with local safety limits.
- Show weather forecasts, crop recommendations and analytics.
- Store telemetry and irrigation events for historical charts and troubleshooting.

3. Architecture

1. Hardware: ESP32 reads sensors, applies local safety rules and controls a compatible relay.
2. Communication: ESP32 sends readings to Flask over Wi-Fi and checks for queued commands.
3. Backend: Flask validates requests, stores records in SQLite, calls weather APIs and serves recommendations.
4. Frontend: HTML/CSS/JavaScript and Chart.js display dashboard cards, forms, charts and alerts.

Telemetry flow: sensors → ESP32/MicroPython → Wi-Fi → Flask API → validation → SQLite → dashboard. Command flow: dashboard → authenticated Flask endpoint → ESP32 retrieves command → local safety checks → relay → pump → acknowledgement/event.

On a local network, configure ESP32 with the laptop’s reachable LAN IP, not 127.0.0.1. Public deployment requires HTTPS and device authentication.

4. Hardware connections and limitations

Part	Purpose / proposed connection	Checks
ESP32	Sensor reading, Wi-Fi and relay control	GPIO inputs are not 5V tolerant.
Capacitive soil sensor	AO to ADC pin such as GPIO34; GND common; VCC per module spec	Check AO voltage and calibrate dry/wet.
DHT22	DATA to a free GPIO (GPIO5 if available); VCC/GND per module	Bare sensors may need a pull-up; some breakout boards include one.
Rain sensor	DO for wet/dry or AO if supported	Confirm active-high/low and output voltage.
Flow sensor	Pulse output to suitable GPIO	Check signal voltage and calibrate pulses/litre.
Relay	ESP32 GPIO to relay input only if 3.3V-compatible	Verify input logic and contact ratings.
Pump + 5V adapter	Pump power switched by correctly rated relay contacts	Adapter current must meet pump requirements.
LED	GPIO through suitable current-limiting resistor	Do not assume an unresisted LED is safe.
Tank-level sensor	Not available in current kit	Show “not monitored”.

Never power the pump from an ESP32 GPIO. Never connect a possibly 5V sensor output directly to an ESP32 input. Verify exact relay, pump and adapter ratings before final wiring; a 5V label alone does not prove the adapter can run the pump safely.

5. Sensor calibration and MicroPython firmware

Soil moisture

Record raw ADC values in dry and wet soil. If raw value decreases as moisture increases: moisture_percent = 100 × (dry_raw – raw) / (dry_raw – wet_raw), then clamp to 0–100. Reverse the formula if sensor behaviour is opposite. This is a calibrated estimate, not laboratory volumetric water content.

DHT22 and rain sensor

Validate readings and report errors explicitly. Determine rain-module polarity experimentally. A rain board detects wetness at its surface; it is not a calibrated rainfall gauge.

Flow sensor

Count pulses over a defined interval. If P is calibrated pulses per litre, estimated litres = pulse_count / P. Calibrate using the specific model and a measured volume. Until then display pulses or “not calibrated”.

Suggested firmware files

```
esp32_firmware/
  main.py          # main loop
  config.py        # pins, thresholds, calibration
  sensors.py       # moisture, DHT22, rain, flow
  wifi_manager.py  # connect/reconnect
  api_client.py    # telemetry and command polling
  irrigation.py    # manual/automatic policy
  safety.py        # runtime limit, faults, startup OFF
```

Keep local safety logic running if Wi-Fi fails. Set pump OFF at startup and enforce a maximum runtime locally on the ESP32.

6. Backend, database and APIs

Recommended stack: Flask REST API, SQLite for a prototype, validated JSON and parameterized SQL or an ORM. Keep irrigation, weather, analytics and ML logic in separate services.

Table	Important fields	Purpose
users	id, username, email, password_hash	Accounts; never store plain-text passwords.
farms	id, user_id, name, location, lat, lon, area, crop, soil_type, water_source	Farm profiles.
devices	id, farm_id, key_hash, last_seen, firmware_version, status	Registered ESP32 devices.
telemetry	id, device_id, timestamp, moisture, temperature, humidity, rain, flow_pulses, pump_state, sensor_status	Historical readings.
schedules	id, farm_id, name, start_time, target_moisture, runtime_limit, status, notes	Planned irrigation.
irrigation_events	id, farm_id, started_at, ended_at, trigger_source, estimated_litres, status, stop_reason	Pump and water history.
commands	id, device_id, type, created_at, delivered_at, acknowledged_at, status	Command lifecycle.
notifications	id, user_id, message, severity, created_at, read_at	Alerts.

Endpoint	Method	Purpose
/api/telemetry	POST	Receive readings.
/api/telemetry/latest	GET	Latest values.
/api/telemetry/history	GET	Chart history.
/api/farms	GET/POST	List/create farms.
/api/farms/<id>	GET/PUT/DELETE	View/update/archive farm.
/api/schedules	GET/POST	List/create schedules.
/api/irrigation/command	POST	Queue authenticated command.
/api/irrigation/events	GET	Irrigation history.
/api/weather	GET	Cached forecast.
/api/advisor/crop	POST	Crop recommendation.
/api/advisor/irrigation	POST	Irrigation advice.
/api/analytics/summary	GET	Aggregated analytics.
/api/notifications	GET	Alerts.

Authenticate users/devices, authorize every farm resource, hash passwords, validate inputs, expire old commands, keep secrets out of source control and use HTTPS publicly.

7. Dashboard features and implementation

Dashboard overview

- Farm/crop/location header and selected device.
- Device online/offline, last-seen timestamp and sensor health.
- Cards for moisture, temperature, humidity, rain and flow status.
- Pump card with Manual/Auto mode, requested state, reported state and runtime.
- Today’s irrigation events, runtime and calibrated estimated water use.
- AI/rule summary explaining inputs and decision reason.
- Command history with request, acknowledgement and final result.

Do not label stale readings as live. Use No data, Stale, Sensor error or Not installed states. Requested pump state and reported pump state must be separate.

Farms

Create/edit/archive farm profiles with name, state/district, location/coordinates, land area and units, crop, soil type, season and water source. Restrict each farm to its authenticated user.

Schedules

Store schedule name, farm, start time, target moisture, maximum runtime, active/paused state and notes. Before running a schedule, verify Auto mode, valid sensors, rain policy, cooldown and runtime limit. Distinguish Scheduled, Requested, Acknowledged and Completed.

Weather

Use farm latitude/longitude with Open-Meteo. Display forecast time, temperature, precipitation probability/amount and retrieval time. Cache responses and warn if data is stale.

AI Crop Advisor

- Easy crop suggestion: location, soil type, season, water source, land area and matching environmental data.
- Advanced N/P/K mode: use only genuine lab, sensor, trusted external or user-entered values, labelled with source.
- Irrigation advice: moisture/trend, crop profile, rain, forecast and recent irrigation.
- Fertilizer guidance must state assumptions and use trustworthy agronomic sources.
- Disease detection is optional and requires image uploads plus an evaluated model.
- Chatbot should use actual farm/weather values and trusted references, not invent facts.

Analytics

- Moisture history and threshold crossings.
- Temperature/humidity trends.
- Irrigation timeline and pump runtime.
- Water usage only after flow calibration.
- Device uptime, missing readings and sensor errors.
- Optional crop-water projections with assumptions documented.

Profile and notifications

Profile can manage username, password, units, timezone, default farm and alerts. Notify on device offline, sensor failure, low moisture, rain, pump timeout, command failure and irrigation completion.

8. Irrigation rules and safety

1. Validate readings and device status.
2. Confirm Auto mode.
3. Check rain state and weather policy.
4. Check minimum off-time/cooldown and maximum runtime.
5. If moisture is below the configured start threshold and all checks pass, request pump ON.
6. Stop at recovery threshold, maximum runtime, fault or emergency stop.
7. Record event, measurements and stop reason.

Use hysteresis to avoid rapid switching. Example only: start below 35%, stop at or above 45%; calibrate for the crop, soil, sensor and installation. These are not universal thresholds.

- Pump defaults OFF on startup and critical faults.
- Invalid critical readings prevent automatic activation.
- Maximum runtime is enforced locally, not only in the cloud.
- Network loss must not disable local safety.
- Emergency stop overrides schedules and AI recommendations.
- Test with pump disconnected before live operation.

9. Datasets and external APIs

Crop recommendation dataset

Candidate dataset: https://data.mendeley.com/datasets/vynxnppr7j/1 . Alternative candidate: https://data.mendeley.com/datasets/7j8cz7p7w9 . Verify availability, licence, provenance, feature definitions and limitations before use.

Common crop datasets contain N, P, K, temperature, humidity, pH, rainfall and crop label. They can support a crop-classification demo, but do not contain live field moisture, pump state, actual farm water use or guaranteed yield. Inputs must match training features; do not fabricate missing values.

Weather API: Open-Meteo

Official docs: https://open-meteo.com/en/docs . Query current/hourly forecast by farm coordinates, cache responses, store retrieval time and display stale-data warnings. Check current usage and attribution requirements.

```
Example request shape; coordinates are illustrative, not the user's farm:
https://api.open-meteo.com/v1/forecast?latitude=17.385&longitude=78.4867&current=temperature_2m,relative_humidity_2m&hourly=precipitation_probability,precipitation&timezone=Asia%2FKolkata
```

SoilGrids

Docs: https://docs.isric.org/globaldata/soilgrids/ . SoilGrids offers mapped soil-property estimates, not measurements from the exact plot. Check current API availability and label results as mapped estimates.

Your own telemetry dataset

Your ESP32 readings are the key dataset for farm-specific analytics. Store timestamp, device/farm ID, moisture estimate, temperature, humidity, rain state, flow pulses/rate if calibrated, pump state and sensor status.

Optional disease image dataset

PlantVillage repository: https://github.com/spMohanty/PlantVillage-Dataset . Review licence and limitations, use a held-out test set, and prevent near-duplicate/source leakage between training and testing.

Feature	Data source	Connection
Live moisture	Capacitive sensor	ESP32 → telemetry API → SQLite → dashboard.
Temperature/humidity	DHT22	ESP32 → API → SQLite → dashboard.
Rain state	Rain sensor	ESP32 → API → dashboard/policy.
Water use	Flow sensor	Pulse count → calibration → event/analytics.
Weather	Open-Meteo	Flask service → cache → weather/advisor.
Crop suggestion	Dataset + matching input features	Form → preprocessing/model → result with limitations.
Soil context	SoilGrids if available	Coordinates → service → labelled estimate.
Analytics	Telemetry/event tables	SQL aggregates → API → Chart.js.
Disease detection	Optional image dataset/model	Image upload → classifier → prediction.
Tank/electricity/NPK/pH	No confirmed current source	Mark unavailable or use sourced input; never fake values.

10. Machine-learning workflow

Crop classifier

1. Download dataset and inspect licence/provenance.
2. Inspect columns, units, missing values, duplicates and class balance.
3. Split into training and held-out test sets; stratify where appropriate.
4. Train a baseline Decision Tree or Random Forest.
5. Evaluate held-out accuracy, precision, recall, F1 and confusion matrix.
6. Check leakage; training accuracy alone is not valid performance evidence.
7. Save preprocessing and model together and version the model.
8. Load model in Flask; validate inputs match training features.
9. Return recommendation and limitations.

Irrigation ML and anomaly detection

Start with transparent rules. Once representative labelled telemetry exists, predict whether moisture may cross a threshold within a defined future window, using time-aware validation. For anomalies, begin with impossible ranges, missing/stuck readings, flow pulses while pump is OFF, and moisture behaviour inconsistent with recorded events.

11. Project directory structure

```
DHARAA/
  esp32_firmware/
    main.py, config.py, sensors.py, wifi_manager.py
    api_client.py, irrigation.py, safety.py
  backend/
    app.py, database.py, models.py, routes/, services/, ml/, data/
  frontend/
    dashboard.html, farms.html, schedules.html
    weather.html, advisor.html, analytics.html, profile.html
    css/, js/
  requirements.txt
  README.md
```

12. Step-by-step roadmap

1. Phase 1: identify exact sensor/relay/adapter models and ratings; test sensors individually.
2. Phase 2: calibrate moisture, confirm rain polarity and determine flow pulses/litre.
3. Phase 3: implement firmware, Wi-Fi reconnect, telemetry and local safety; test with pump disconnected.
4. Phase 4: build Flask app, SQLite schema, telemetry and history endpoints.
5. Phase 5: send ESP32 readings and verify records/reconnect behaviour.
6. Phase 6: build dashboard cards, stale states, history and charts.
7. Phase 7: implement farm/schedule CRUD, command lifecycle and event history.
8. Phase 8: verify electrical setup, then implement manual/automatic control, hysteresis, timeout and emergency stop.
9. Phase 9: integrate Open-Meteo and caching.
10. Phase 10: prepare dataset, train/evaluate crop model and expose inference endpoint.
11. Phase 11: add analytics and calibrated water estimates.
12. Phase 12: add authentication, HTTPS, secrets management, logging, backups and recovery.

Recommended order: reliable sensor readings → Flask/SQLite → dashboard → safe pump control → weather → crop model → advanced analytics.

13. Testing checklist

- Moisture changes consistently from dry to wet soil.
- DHT22 readings are valid and errors are visible.
- Rain sensor polarity/threshold are confirmed.
- Flow pulses are counted; litres are withheld until calibrated.
- Invalid values are not replaced by fabricated values.
- Old readings are marked stale.
- Backend validates telemetry and associates farms correctly.
- Requested and reported pump states are distinct; commands expire and acknowledgements are recorded.
- Charts handle empty data and weather shows retrieval time.
- Test startup OFF, timeout, cooldown, sensor failure, network loss and emergency stop.
- Evaluate ML on held-out data and document features, units and limitations.

14. Deployment and maintenance

- Keep passwords/API keys out of source control.
- Hash passwords and authorize farm/device operations.
- Use HTTPS and authenticated device requests publicly.
- Keep pump safety independent of cloud availability.
- Back up the database and test restoration.
- Record firmware/model versions and log errors without exposing secrets.
- Check dataset licences and API usage/attribution requirements.
- Keep water and electrical components separated and protected.

A laptop-hosted Flask server is suitable for local prototyping. Remote deployment needs persistent storage and a suitable scheduler. SQLite on an ephemeral filesystem may lose data after restart/redeploy.

15. Feature availability matrix

Feature	Status with current kit	Dependency
Dashboard/farms/telemetry	Implementable	ESP32, Flask, database, frontend.
Schedules	Implementable	Scheduler and safety checks.
Weather	Implementable	Internet and weather API.
Manual pump control	After electrical verification	Compatible relay and rated adapter.
Automatic irrigation	After calibration/testing	Local safety and calibrated moisture.
Crop recommendation	Optional	Dataset and matching inputs.
Historical analytics	As data accumulates	Telemetry/event history.
Water-volume analytics	After calibration	Flow calibration factor.
Tank level	Unavailable now	Tank-level sensor required.
Electricity consumption	Unavailable now	Power/energy measurement required.
NPK/pH	Not measured now	Suitable sensor, lab or trusted data.
Multiple depths/zones	Unavailable now	Additional sensors/devices.
Disease detection	Optional	Images, dataset, evaluated model.

Conclusion: build sensing, communication, storage, dashboard and safe control first. Weather APIs and public datasets add context; your own telemetry powers farm-specific analytics. Distinguish measured values, estimates, forecasts and ML predictions.